

Simulazione Molecolare di Materiali

Giuseppe Milano¹ Antonio De Nicola¹

A.A. 2026/27

¹Università di Napoli, Federico II - DiCMaPI Piazz.le Tecchio, Napoli, Italy

`giuseppe.milano@unina.it   antonio.denicola@unina.it`

Lab 2 - VERLET ALGORITHM AND PERIODIC BOUNDARY CONDITIONS

Solution of Newton's Eqs. of motion - Verlet Algorithm

Let's expand the particle coordinate r in time interval t

$$r(t + \Delta t) = r(t) + \frac{dr}{dt}\Delta t + \frac{1}{2}\frac{d^2r}{dt^2}\Delta t^2 + \frac{1}{3!}\frac{d^3r}{dt^3}\Delta t^3 + O(\Delta t^4)$$

Solution of Newton's Eqs. of motion - Verlet Algorithm

Let's expand the particle coordinate r in time interval t

$$r(t + \Delta t) = r(t) + \frac{dr}{dt}\Delta t + \frac{1}{2}\frac{d^2r}{dt^2}\Delta t^2 + \frac{1}{3!}\frac{d^3r}{dt^3}\Delta t^3 + O(\Delta t^4)$$

from *Newton's Law*: $\vec{F} = m \times \vec{a} = m \frac{d^2r}{dt^2}$; $\frac{d^2r}{dt^2} = \frac{\vec{F}}{m}$

- First derivatives of position r over the time t is defined as velocity v .
- Second derivatives of position r over the time t is the ratio: $\frac{\vec{F}}{m} = \vec{a}$

Solution of Newton's Eqs. of motion - Verlet Algorithm

Let's expand the particle coordinate r in time interval t

$$r(t + \Delta t) = r(t) + \frac{dr}{dt}\Delta t + \frac{1}{2}\frac{d^2r}{dt^2}\Delta t^2 + \frac{1}{3!}\frac{d^3r}{dt^3}\Delta t^3 + O(\Delta t^4)$$

from *Newton's Law*: $\vec{F} = m \times \vec{a} = m \frac{d^2r}{dt^2}$; $\frac{d^2r}{dt^2} = \frac{\vec{F}}{m}$

- First derivatives of position r over the time t is defined as velocity v .
- Second derivatives of position r over the time t is the ratio: $\frac{\vec{F}}{m} = \vec{a}$

Taylor Expansion

$$f(x_0 + x) = f(x_0) + \frac{x}{1!}f'(x_0) + \frac{x^2}{2!}f''(x_0) + \dots + \frac{x^n}{n!}f^{(n)}(x_0)$$

Note: The function and its derivatives up to order n must be continuous.

Solution of Newton's Eqs. of motion - Verlet Algorithm

$$r(t + \Delta t) = r(t) + \frac{dr}{dt}\Delta t + \frac{1}{2}\frac{d^2r}{dt^2}\Delta t^2 + \frac{1}{3!}\frac{d^3r}{dt^3}\Delta t^3 + O(\Delta t^4)$$

Solution of Newton's Eqs. of motion - Verlet Algorithm

$$r(t + \Delta t) = r(t) + \frac{dr}{dt}\Delta t + \frac{1}{2}\frac{d^2r}{dt^2}\Delta t^2 + \frac{1}{3!}\frac{d^3r}{dt^3}\Delta t^3 + O(\Delta t^4)$$

$$r(t + \Delta t) = r(t) + v\Delta t + \frac{f}{2m}\Delta t^2 + \frac{1}{3!}\frac{d^3r}{dt^3}\Delta t^3 + O(\Delta t^4)$$

Solution of Newton's Eqs. of motion - Verlet Algorithm

$$r(t + \Delta t) = r(t) + \frac{dr}{dt}\Delta t + \frac{1}{2}\frac{d^2r}{dt^2}\Delta t^2 + \frac{1}{3!}\frac{d^3r}{dt^3}\Delta t^3 + O(\Delta t^4)$$

$$r(t + \Delta t) = r(t) + v\Delta t + \frac{f}{2m}\Delta t^2 + \frac{1}{3!}\frac{d^3r}{dt^3}\Delta t^3 + O(\Delta t^4)$$

similarly

$$r(t - \Delta t) = r(t) - v\Delta t + \frac{f}{2m}\Delta t^2 - \frac{1}{3!}\frac{d^3r}{dt^3}\Delta t^3 + O(\Delta t^4)$$

Solution of Newton's Eqs. of motion - Verlet Algorithm

$$r(t + \Delta t) = r(t) + \frac{dr}{dt}\Delta t + \frac{1}{2}\frac{d^2r}{dt^2}\Delta t^2 + \frac{1}{3!}\frac{d^3r}{dt^3}\Delta t^3 + O(\Delta t^4)$$

$$r(t + \Delta t) = r(t) + v\Delta t + \frac{f}{2m}\Delta t^2 + \frac{1}{3!}\frac{d^3r}{dt^3}\Delta t^3 + O(\Delta t^4)$$

similarly

$$r(t - \Delta t) = r(t) - v\Delta t + \frac{f}{2m}\Delta t^2 - \frac{1}{3!}\frac{d^3r}{dt^3}\Delta t^3 + O(\Delta t^4)$$

by summing the two terms

$$r(t + \Delta t) + r(t - \Delta t) = 2r(t) + \frac{f(t)}{m}\Delta t^2 + O(\Delta t^4)$$

$$r(t + \Delta t) \approx 2r(t) - r(t - \Delta t) + \frac{f(t)}{m}\Delta t^2$$

Solution of Newton's Eqs. of motion - Verlet Algorithm

$$r(t + \Delta t) \approx 2r(t) - r(t - \Delta t) + \frac{f(t)}{m} \Delta t^2$$

Solution of Newton's Eqs. of motion - Verlet Algorithm

$$r(t + \Delta t) \approx 2r(t) - r(t - \Delta t) + \frac{f(t)}{m} \Delta t^2$$

position of r at time $t, t - \Delta t$

forces acting on the particle (r) at time t

Solution of Newton's Eqs. of motion - Verlet Algorithm

$$r(t + \Delta t) \approx 2r(t) - r(t - \Delta t) + \frac{f(t)}{m} \Delta t^2$$

position of r at time $t, t - \Delta t$

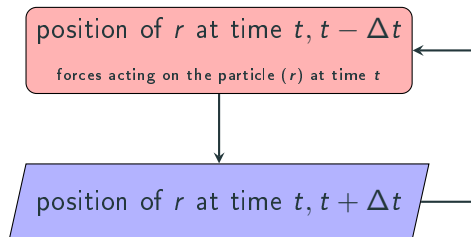
forces acting on the particle (r) at time t



position of r at time $t, t + \Delta t$

Solution of Newton's Eqs. of motion - Verlet Algorithm

$$r(t + \Delta t) \approx 2r(t) - r(t - \Delta t) + \frac{f(t)}{m} \Delta t^2$$



Trajectory of a projectile

Let's apply the Verlet Algorithm to a simple case, the calculation of the trajectory (also known as flight path) of an object under the action of the gravity.

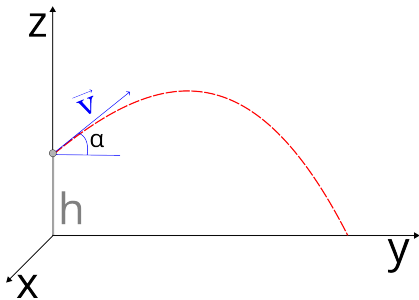


Figure 1: trajectory of a projectile

Initial Conditions, time $t = 0$

$$a_x = 0$$

$$a_y = 0$$

$$a_z = -g$$

Trajectory of a projectile

Let's apply the Verlet Algorithm to a simple case, the calculation of the trajectory (also known as flight path) of an object under the action of the gravity.

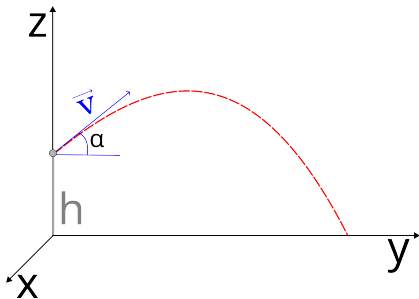


Figure 1: trajectory of a projectile

Initial Conditions, time $t = 0$

$$a_x = 0$$

$$a_y = 0$$

$$a_z = -g$$

$$v_x(0) = 0; x(0) = 0$$

$$v_y(0) = v_0 \cos(\alpha); y(0) = 0$$

$$v_z(0) = v_0 \sin(\alpha); z(0) = h$$

Trajectory of a projectile

Note

By integrating the a_x, a_y, a_z we get the expression of the three components x, y, z of the velocity.

hence:

$$a_x = 0 \rightarrow v_x = c_x$$

$$a_y = 0 \rightarrow v_y = c_y$$

$$a_z = -g \rightarrow v_z = -gt + c_z$$

Trajectory of a projectile

Note

By integrating the a_x, a_y, a_z we get the expression of the three components x, y, z of the velocity.

hence:

$$a_x = 0 \rightarrow v_x = c_x$$

$$a_y = 0 \rightarrow v_y = c_y$$

$$a_z = -g \rightarrow v_z = -gt + c_z$$

$$v_x = 0$$

$$v_y = v_0 \cos(\alpha)$$

$$v_z = -gt + v_0 \sin(\alpha)$$

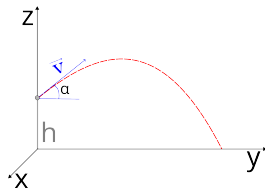
Trajectory of a projectile

Integrating the velocities over the time t , and considering the initial conditions, the final expressions of $x(t)$, $y(t)$, $z(t)$ are:

$$x = 0$$

$$y = (v_0 \cos(\alpha))t$$

$$z = -\frac{1}{2}gt^2 + v_0 \sin(\alpha)t + h$$



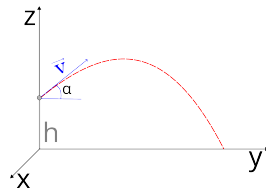
Trajectory of a projectile

Integrating the velocities over the time t , and considering the initial conditions, the final expressions of $x(t)$, $y(t)$, $z(t)$ are:

$$x = 0$$

$$y = (v_0 \cos(\alpha))t$$

$$z = -\frac{1}{2}gt^2 + v_0 \sin(\alpha)t + h$$



The expression of the trajectory z is obtained by expressing z as function of the time position y :

$$z(y) = -\frac{1}{2}g \frac{y^2}{v_0^2 \cos^2(\alpha)} + \tan(\alpha)y + h$$

Trajectory of a projectile

The example reported above represents a simple problem where the solution exists and is analytic.

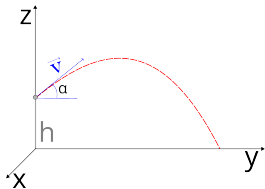


Figure 2: Simple case with an analytic solution.

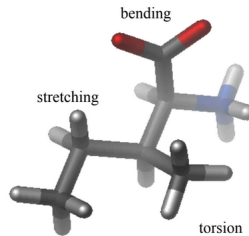
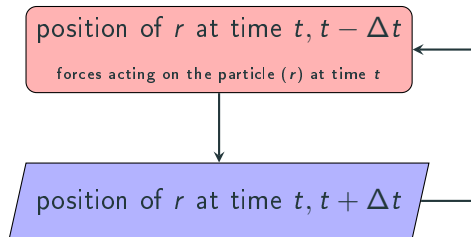


Figure 3: Multi-body problem

Even a single molecule represents a multi-body problem !

Solution of Newton's Eqs. of motion - Verlet Algorithm

$$r(t + \Delta t) \approx 2r(t) - r(t - \Delta t) + \frac{f(t)}{m} \Delta t^2$$



The sequence of all *iterations* composes the *trajectory* of the particle !

Exercise 1: Verlet Algorithm - Practical Application

Hands-on

You must edit the file *proiettile.f* in the *LAB-2* directory (editor: *VIM*).

Exercise 1: Verlet Algorithm - Practical Application

Hands-on

You must edit the file *proiettile.f* in the *LAB-2* directory (editor: *VIM*).

```
Program Proiettile
real mass
!### Variable Definition
tstep = 0.01
mass = 1.0
alpha = 30.0
g = 9.81
nstep = 5000
v0 = 120.0
```

```
!### Conversion degree to rad.
alpha = alpha * 0.01745
!### Initial Position Assignment
rx0 = 0.0
ry0 = 0.0
rz0 = 8.0
rx = rx0
ry = ry0
rz = rz0
```

Exercise 1: Verlet Algorithm - Practical Application

Hands-on

You must edit the file *proiettile.f* in the *LAB-2* directory (editor: *VIM*).

```
Program Proiettile
real mass
!### Variable Definition
tstep = 0.01
mass = 1.0
alpha = 30.0
g = 9.81
nstep = 5000
v0 = 120.0
```

```
!### Conversion degree to rad.
alpha = alpha * 0.01745
!### Initial Position Assignment
rx0 = 0.0
ry0 = 0.0
rz0 = 8.0
rx = rx0
ry = ry0
rz = rz0
```

> continue on the next slide ...

Exercise 1: Verlet Algorithm - Practical Application

!### Initial velocities Assignment

`vx0 = 0.0`

`vy0 = v0 * cos(alpha)`

`vz0 = v0 * sin(alpha)`

`vxx = vx0`

`vyy = vy0`

`vzz = vz0`

!### Computation of particle at time t-dt

`rx = rx - timestep*vxx`

`ry = ry - timestep*vyy`

`rz = rz - timestep*vzz`

!### Main Loop

`do i = 1, nstep`

!### Computation of the force components at t

`fxx = 0.0`

`fyy = 0.0`

`fzz = -mass * g`

!### Integration Eqs. of motions

`xx = 2.0 * rx - rxf + timestep**2 * (fxx/mass)`

`yy = 2.0 * ry - ryf + timestep**2 * (fyy/mass)`

`zz = 2.0 * rz - rzf + timestep**2 * (fzz/mass)`

> continue on the next slide ...

Exercise 1: Verlet Algorithm - Practical Application

```
!### Propagation of velocity v
vxx = (xx - rxf) / (2.d0*tstep)
vyy = (yy - ryf) / (2.d0*tstep)
vzz = (zz - rzf) / (2.d0*tstep)
!### Update of the position at time t-dt
rx = rx
ry = ry
rz = rz

!### Update to current positions

rx = xx
ry = yy
rz = zz
!### Stop when the projectile hits the ground
if (zz.lt.0.0) exit
!### Trajectory Writing
write(6,*) yy, zz
enddo
stop
end
```

Exercise 1: Verlet Algorithm - Practical Application

How to execute the code

- save the file with the name *proiettile.f*
- compile the code with the command: `gfortran -o proiettile proiettile.f`
- execute the code with the command: `./proiettile > out1.dat`

Exercise 1: Verlet Algorithm - Practical Application

How to execute the code

- save the file with the name *proiettile.f*
- compile the code with the command: *gfortran -o proiettile proiettile.f*
- execute the code with the command: *./proiettile > out1.dat*

Visualize the trajectory

```
gnuplot> plot 'out1.dat' w lp
```

What do you obtained ?

Verlet Algorithm - Elastic Collision

IF Statement

The block **IF** statement evaluates a logical expression and, if the logical expression is true, it executes a set of statements called the IF block.

```
Program IF_EXAMPLE
```

```
i = 3
```

```
if (i.lt.5) then
```

```
print *, i, "is lower than 5"
```

```
else
```

```
print *, i, "is greater than 5"
```

```
endif
```

```
stop
```

```
end
```

IF Directives

.lt. (lower than)

.gt. (greater that)

.eq. (equal to)

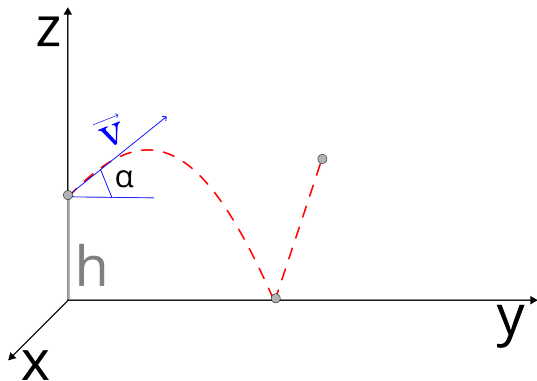
.ge. (greater equal to)

.le. (lower equal to)



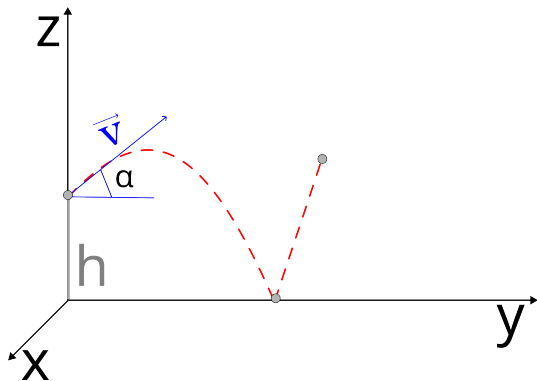
Verlet Algorithm - Elastic Collision

Let's include the effect of an *elastic collision* to our code by using the statement *IF*.



Verlet Algorithm - Elastic Collision

Let's include the effect of an *elastic collision* to our code by using the statement *IF*.



How to do it ? We should slightly modify the program [urto.f](#)

Velocity Verlet Algorithm

$$r(t + \Delta t) = r(t) + v(t)\Delta t + \frac{f(t)}{2m}\Delta t^2$$

$$v(t + \Delta t) = v(t) + \frac{f(t) + f(t + \Delta t)}{2m}\Delta t$$

Velocity Verlet Algorithm

$$r(t + \Delta t) = r(t) + v(t)\Delta t + \frac{f(t)}{2m}\Delta t^2$$

$$v(t + \Delta t) = v(t) + \frac{f(t) + f(t + \Delta t)}{2m}\Delta t$$

`urto.f`

The program *urto.f* is written with the *Velocity Verlet*: it has the same structure and the same variables of *proiettile.f*, but the velocities (v_{xx} , v_{yy} , v_{zz}) are propagated explicitly.

Velocity Verlet Algorithm

$$r(t + \Delta t) = r(t) + v(t)\Delta t + \frac{f(t)}{2m}\Delta t^2$$
$$v(t + \Delta t) = v(t) + \frac{f(t) + f(t + \Delta t)}{2m}\Delta t$$

urto.f

The program *urto.f* is written with the *Velocity Verlet*: it has the same structure and the same variables of *proiettile.f*, but the velocities (v_{xx} , v_{yy} , v_{zz}) are propagated explicitly.

Collisions

The velocity is part of the algorithm: at a collision we can modify it directly!

Exercise 2: Verlet Algorithm - Practical Application

Steps to do

- copy the source file: `cp ../urto.f .`
- modify `urto.f` (insert the IF block)
- compile the code. `gfortran -o urto.x urto.f`
- execute the code: `./urto.x > out2.dat`

Exercise 2: Verlet Algorithm - Practical Application

Steps to do

- copy the source file: `cp ../urto.f .`
- modify `urto.f` (insert the IF block)
- compile the code. `gfortran -o urto.x urto.f`
- execute the code: `./urto.x > out2.dat`

IF block to include in the code

(the file can be edited with VIM)

```
!### INSERT IF BLOCK HERE
if (rz.lt.0.0 .and. vzz.lt.0.0) then
    vyy = vyy
    vzz = -vzz
endif
```

Exercise 2: Verlet Algorithm - Practical Application

IF block to include in the code

(the file can be edited with VIM)

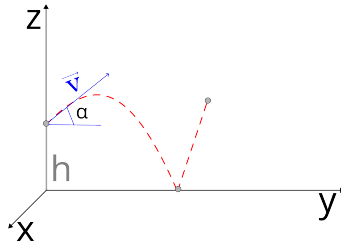
```
!### INSERT IF BLOCK HERE
if (rz.lt.0.0 .and. vzz.lt.0.0) then
    vyy = vyy
    vzz = -vzz
endif
```

Exercise 2: Verlet Algorithm - Practical Application

IF block to include in the code

(the file can be edited with VIM)

```
#### INSERT IF BLOCK HERE
if (rz.lt.0.0 .and. vzz.lt.0.0) then
    vyy = vyy
    vzz = -vzz
endif
```



The sign of the velocity is inverted on Z component, while is unchanged on Y .

Attention

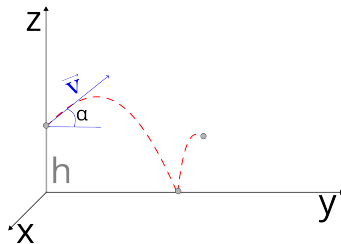
$z = 0$ is never hit exactly: we test $z < 0$ and $v_z < 0$ (moving down).

Exercise 3: Verlet Algorithm - Practical Application

IF block to include in the code

Partial Elastic Collision

```
!### INSERT IF BLOCK HERE  
if (rz.lt.0.0 .and. vzz.lt.0.0) then  
    vyy = 0.9*vyy  
    vzz = -0.9*vzz  
endif
```



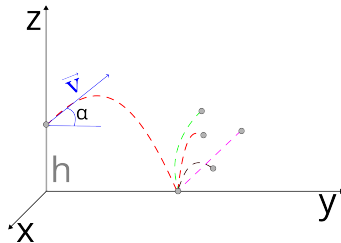
Magnitude of the velocity is reduced by a factor 0.9.

Exercise 4: Verlet Algorithm - Practical Application

IF block to include in the code

Random Scattering

```
!### INSERT IF BLOCK HERE  
if (rz.lt.0.0) then  
    vzz = -2.0 * rand() * vzz  
endif
```



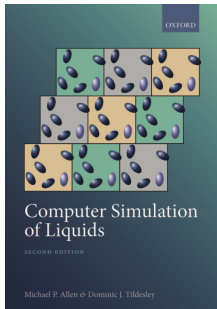
Each bounce redraws v_{zz} from the fixed launch speed $vz0$: heights stay random and visible, without decaying or blowing up.

Random Numbers in Fortran

the intrinsic function `rand()` extract, randomly, a number between 0 and 1.

Periodic Boundary Conditions

Periodic Boundary Conditions (PBC)



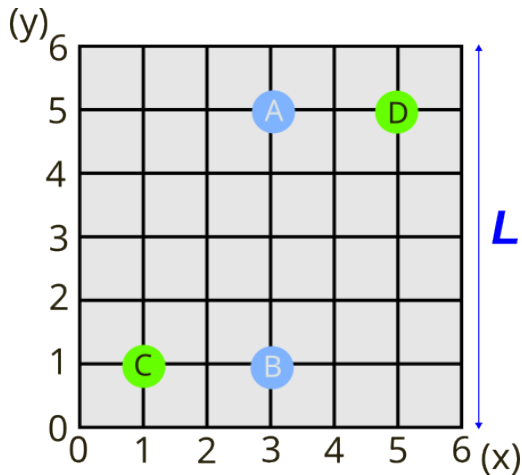
1.6.4 Computer code for periodic boundaries

How do we handle periodic boundaries and the minimum image convention in a simulation program? Let us assume that, initially, the N molecules in the simulation lie within a cubic box of side L , with the origin at its centre, that is, all coordinates lie in the range $(-\frac{1}{2}L, \frac{1}{2}L)$. As the simulation proceeds, these molecules move about the infinite periodic system. When a molecule leaves the box by crossing one of the boundaries, it is usual to switch attention to the image molecule entering the box by simply adding L to, or subtracting L from, the appropriate coordinate. One simple way to do this uses an IF statement to test the positions immediately after the molecules have been moved (whether by MC or MD). For example,

```
IF ( r(1,i) > box2 ) r(1,i) = r(1,i) - box
```

```
IF ( r(1,i) < -box2 ) r(1,i) = r(1,i) + box
```

Periodic Boundary Conditions (PBC)

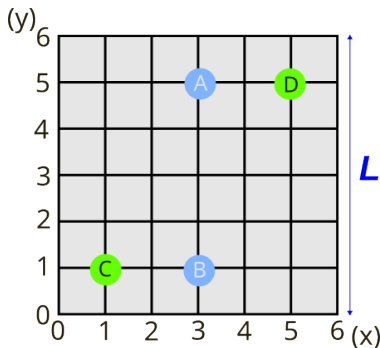


Let's consider the coordinates of pnts (A,B,C,D) on bi-dimensional a grid:

- A (3, 5)
- B (3, 1)
- C (1, 1)
- D (5, 5)

where $\overline{AB}(y) = \sqrt{(y_B - y_A)^2}$

Periodic Boundary Conditions (PBC)



Let's consider the coordinates of pnts (A,B,C,D) on bi-dimensional a grid:

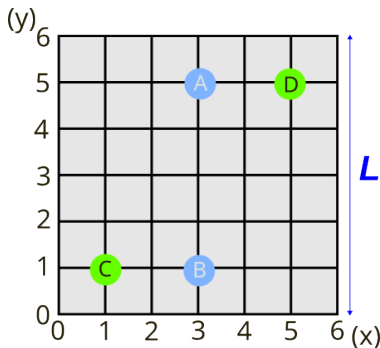
- A (3, 5)
- B (3, 1)
- C (1, 1)
- D (5, 5)

from PBC definition

IF $r(1, i) > box2$

then $r(1, i) = r(1, i) - box$

Periodic Boundary Conditions (PBC)



Let's consider the coordinates of pnts (A,B,C,D) on bi-dimensional a grid:

- A (3,5)
- B (3,1)
- C (1,1)
- D (5,5)

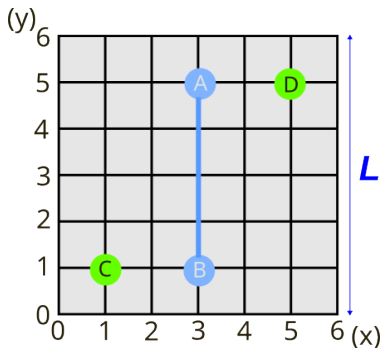
$$\overline{AB}(y) = \sqrt{(y_B - y_A)^2} = 4$$

from PBC definition

IF $r(1,i) > \text{box2}$

then $r(1,i) = r(1,i) - \text{box}$

Periodic Boundary Conditions (PBC)



Let's consider the coordinates of pnts (A,B,C,D) on bi-dimensional a grid:

- A (3,5)
- B (3,1)
- C (1,1)
- D (5,5)

$$\overline{AB}(y) = \sqrt{(y_B - y_A)^2} = 4$$

from PBC definition

IF $r(1,i) > box2$

then $r(1,i) = r(1,i) - box$

Periodic Boundary Conditions (PBC)

The coordinates of pnts A (3,5), and B (3,1). Let's apply the PBC:

Periodic Boundary Conditions (PBC)

The coordinates of pnts A (3,5), and B (3,1). Let's apply the PBC:

IF

$$\overline{AB}(y) > \frac{L}{2}(3)$$

$$\overline{AB}(y) = |\overline{AB}(y) - L| = |4 - 6| = 2$$

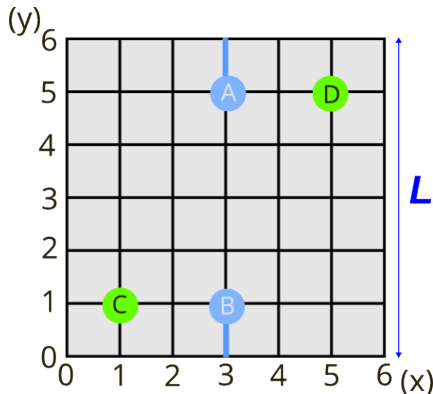
Periodic Boundary Conditions (PBC)

The coordinates of pnts A (3,5), and B (3,1). Let's apply the PBC:

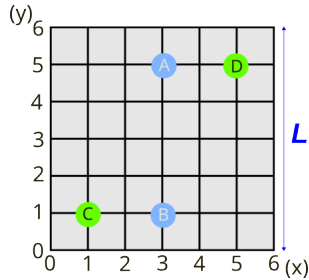
IF

$$\overline{AB}(y) > \frac{L}{2}(3)$$

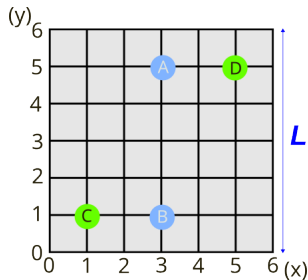
$$\overline{AB}(y) = |\overline{AB}(y) - L| = |4 - 6| = 2$$



Periodic Boundary Conditions (PBC)



Periodic Boundary Conditions (PBC)



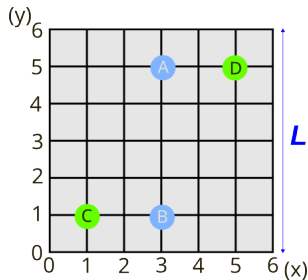
minimum image convention

An atom interacts only with the closest periodic image of any other atom!

Cutoff distance for interactions:

$$r_{cut} = \frac{1}{2} \text{ of cell length}$$

Periodic Boundary Conditions (PBC)

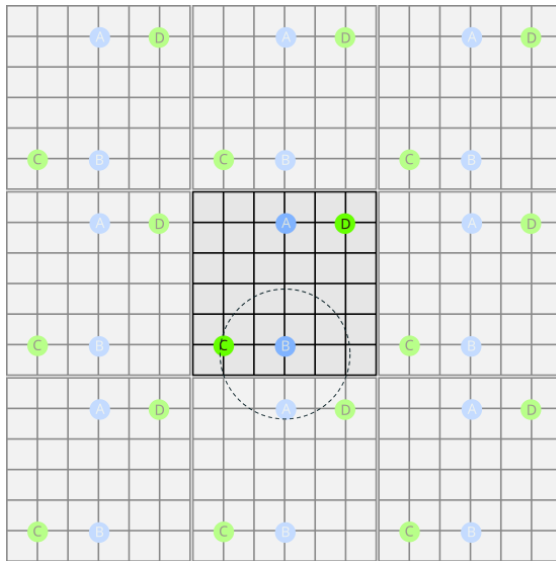


minimum image convention

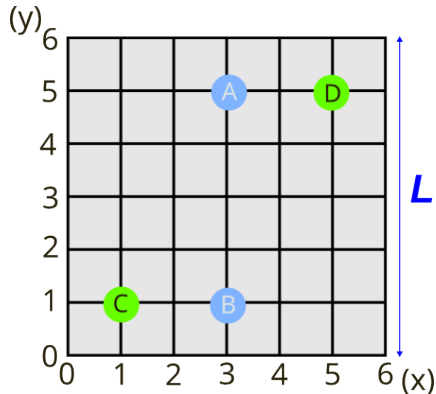
An atom interacts only with the closest periodic image of any other atom!

Cutoff distance for interactions:

$$r_{cut} = \frac{1}{2} \text{ of cell length}$$

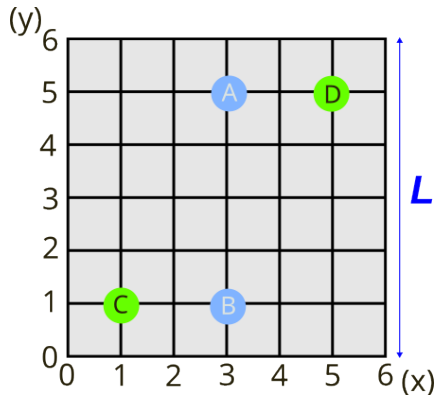


Periodic Boundary Conditions (PBC)



The coordinates of pnts C (1,1), and D (5,5). The X,Y distances are:

Periodic Boundary Conditions (PBC)

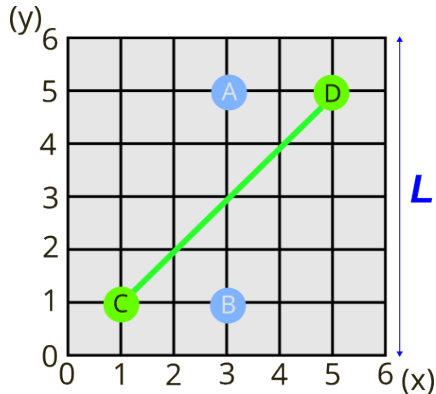


The coordinates of pnts C (1,1), and D (5,5). The X,Y distances are:

$$\overline{CD}(y) = \sqrt{(y_D - y_C)^2}$$

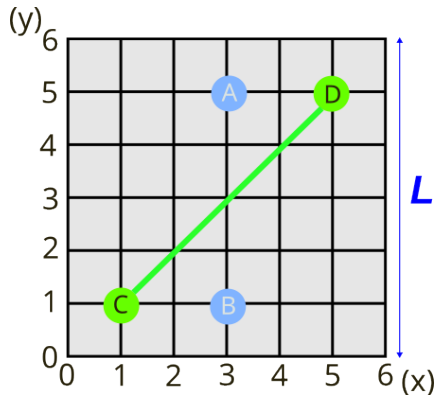
$$\overline{CD}(x) = \sqrt{(x_D - x_C)^2}$$

Periodic Boundary Conditions (PBC)



The coordinates of pnts C (1,1), and D (5,5). The X,Y distances are:

Periodic Boundary Conditions (PBC)



The coordinates of pnts C (1,1), and D (5,5). The X,Y distances are:

$$\overline{CD}(y) = \sqrt{(y_D - y_C)^2} = 4$$

$$\overline{CD}(x) = \sqrt{(x_D - x_C)^2} = 4$$

Periodic Boundary Conditions (PBC)

After PBC are applied, which are the X,Y distances?

$$\overline{CD}(y) = ?$$

$$\overline{CD}(x) = ?$$

